Лекция 3: Полиморфизм. Абстрактные классы, интерфейсы и протоколы. Duck Typing и строгие контракты

Курс: Объектно-ориентированное программирование 1 (CS2203)
Университет: КазНУ им. аль-Фараби
Преподаватель: Санжар Аубакиров, PhD
Аудитория: Бакалавриат, 2-й курс, специальность «6B06110 Программная инженерия»
Формат: Лекция, 2 академических часа (90 минут)
Методологический нарратив: Structure → Order (От фиксированных типов к полиморфным контрактам)

Введение и инженерный контекст

На предыдущих занятиях мы научились изолировать данные внутри классов (инкапсуляция) и соединять объекты через компоненты (композиция).
Сегодня мы переходим к третьему столпу ООП — **полиморфизму** (*Polymorphism*).
Без полиморфизма любая попытка расширить систему новыми типами приводит к бесконечным каскадам проверок типов:

```
# АНТИПАТТЕРН: процедурные проверки типов (Code Smell: Switch on Type)
if isinstance(payment, QrPayment):
    process_qr(payment)
elif isinstance(payment, NfcCardPayment):
    process_nfc(payment)
elif isinstance(payment, BankCardPayment):
    process_bank(payment)
```

Каждый новый способ оплаты заставляет программиста лезть во все файлы проекта и дописывать новые ветки elif.
Полиморфизм устраняет эту хрупкость: вызывающий код отправляет единое абстрактное сообщение («обработай транзакцию»), а конкретный объект сам решает, как именно его выполнить в зависимости от своей внутренней структуры.

План лекции (90 минут)

№	Время	Тема	Формат
1	00:00–00:20	Сущность полиморфизма: динамический vs статический полиморфизм, диспетчеризация вызовов	Теория
2	00:20–00:40	Утиная типизация (Duck Typing) в Python: плюсы,	Анализ кейсов

№	Время	Тема	Формат
		риски и рантайм-сбои	
3	00:40–01:05	Явные контракты: Абстрактные базовые классы (abc.ABC) против Протоколов (typing.Protocol)	Практика + сравнение
4	01:05–01:25	Архитектурный паттерн: Полиморфный диспетчер событий (Event Dispatcher)	Реализация системы
5	01:25–01:30	Заключение, связь с принципом Open/Closed (OCP) и контрольные вопросы	Рефлексия

---

1. Что такое полиморфизм? Динамическая диспетчеризация

В программировании полиморфизм означает способность различных объектов с разной внутренней реализацией предоставлять **одинаковый внешний интерфейс** для взаимодействия.

Виды полиморфизма:

1. **Статический полиморфизм (Compile-time / Ad-hoc / Parametric)**:
   - Перегрузка функций (Function Overloading, характерна для C++, Java).
   - Обобщенное программирование (Generics / Templates).
2. **Динамический полиморфизм (Subtyping / Runtime)**:
   - Разрешение метода происходит во время выполнения программы на основе реального типа объекта в куче (Dynamic Dispatch).

В динамических языках (таких как Python) весь полиморфизм по умолчанию является динамическим: интерпретатор ищет имя метода в словаре объекта __dict__ и его MRO во время выполнения.

---

2. Утиная типизация (Duck Typing) и её границы

Принцип утиной типизации сформулирован Джеймсом Уиткомбом Райли: > *«Если это выглядит как утка, плавает как утка и крякает как утка, то это, вероятно, утка и есть».*

В процедурном или раннем Python разработчики полагались исключительно на наличие метода:

```
def process_refund(payment_source, amount):
    # Код не волнует, является ли payment_source картой, кошельком или ваучером.
```

```
    # Главное — наличие метода refund()
    payment_source.refund(amount)
```

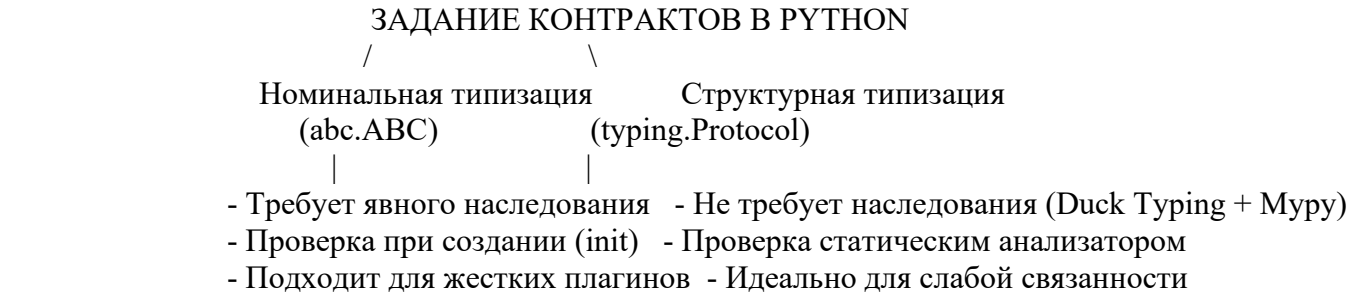
Проблемы неявной утиной типизации в enterprise-разработке:

1. **Отсутствие проверки сигнатур до рантайма**: если класс реализовал метод refund(self, amount, reason), а вызывающий код передал только amount, программа упадет с TypeError в продакшене.
2. **Отсутствие документации контракта**: IDE и статические анализаторы (mypy, pyright) не могут подсказать разработчику правильные аргументы.
3. **Ложные совпадения имен**: метод .close() у файла закрывает файловый дескриптор, а метод .close() у сделки в CRM переводит её в статус «завершена». Совпадение имени не означает совпадение семантики!

---

3. Явные контракты: ABC против Protocol

Для предотвращения скрытых ошибок современный Python 3.12 предлагает два мощных инструмента задания контрактов:



3.1. Абстрактные базовые классы (Nominal Subtyping: abc.ABC)

Класс явно объявляет, что он обязуется реализовать методы базового класса:

```
from abc import ABC, abstractmethod
from decimal import Decimal


class PaymentGateway(ABC):
    """Абстрактный контракт платежного шлюза."""

    @abstractmethod
    def charge(self, account_id: str, amount: Decimal) -> str:
        """Списание средств. Возвращает transaction_id."""
        pass

    @abstractmethod
    def refund(self, transaction_id: str) -> bool:
        """Возврат транзакции."""
        pass


class KaspiQrGateway(PaymentGateway):
    def charge(self, account_id: str, amount: Decimal) -> str:
        # Конкретная реализация для Kaspi API
        return f"KASPI-TX-{account_id[:4]}-998"
```

```
    def refund(self, transaction_id: str) -> bool:
        return True
```
Если класс KaspiQrGateway забудет реализовать хотя бы один абстрактный метод, попытка создать его экземпляр вызовет ошибку TypeError: Can't instantiate abstract class... в момент вызова конструктора!

3.2. Протоколы (Structural Subtyping: typing.Protocol — PEP 544)

Протокол задает контракт **без необходимости явного наследования**:

```
from typing import Protocol
from decimal import Decimal


class Refundable(Protocol):
    """Любой объект, поддерживающий операцию возврата."""
    def refund(self, transaction_id: str) -> bool:
        ...


def execute_emergency_refund(service: Refundable, tx_id: str) -> None:
    success = service.refund(tx_id)
    print(f"Возврат {tx_id}: {'Успешно' if success else 'Ошибка'}")
```
Любой сторонний класс, у которого есть метод refund с совпадающей сигнатурой, автоматически считается совместимым с протоколом Refundable для анализатора mypy.

---

4. Архитектурный кейс: Полиморфный диспетчер событий (Event Dispatcher)

В валидаторе системы Avtobys происходят события разного рода: - PassengerTappedEvent (пассажир приложил карту); - LowBalanceAlertEvent (баланс подошел к концу); - HardwareFailureEvent (сбой сканера QR или термопринтера).
Вместо монолитных условий мы строим полиморфный диспетчер событий:

```
# events/dispatcher.py
from dataclasses import dataclass
from datetime import datetime
from typing import Protocol, Type, Dict, List


@dataclass(frozen=True)
class TransitEvent:
    event_id: str
    timestamp: datetime


@dataclass(frozen=True)
class TapEvent(TransitEvent):
    bus_id: str
    card_id: str


@dataclass(frozen=True)
class ErrorEvent(TransitEvent):
```

```
    error_code: int
    message: str


class EventHandler(Protocol):
    """Полиморфный обработчик событий."""
    def handle(self, event: TransitEvent) -> None:
        ...


class EventDispatcher:
    """Шина диспетчеризации событий валидатора."""
    def __init__(self):
        self._handlers: Dict[Type[TransitEvent], List[EventHandler]] = {}

    def subscribe(self, event_type: Type[TransitEvent], handler: EventHandler) -> None:
        if event_type not in self._handlers:
            self._handlers[event_type] = []
        self._handlers[event_type].append(handler)

    def publish(self, event: TransitEvent) -> None:
        event_type = type(event)
        handlers = self._handlers.get(event_type, [])
        for handler in handlers:
            handler.handle(event)
```

Добавление новых обработчиков без изменения ядра:

```
class AuditLogHandler:
    def handle(self, event: TransitEvent) -> None:
        print(f"[AUDIT LOG] {event.timestamp.isoformat()} : Event {event.event_id}")

class MetricsCollectorHandler:
    def handle(self, event: TransitEvent) -> None:
        # Увеличение счетчика метрик Prometheus
        print(f"[METRIC] Incrementing counter for {type(event).__name__}")
```

Этот дизайн полностью полиморфен и строго соблюдает принцип открытости/закрытости (OCP).

Контрольные вопросы и выводы

1. Чем динамическая диспетчеризация методов отличается от статической перегрузки функций?
2. Почему антипаттерн «Switch on Type» (isinstance) разрушает архитектуру при развитии системы?
3. В каких ситуациях в Python следует предпочесть abc.ABC, а в каких — typing.Protocol?
4. Как полиморфизм позволяет разрабатывать плагинные архитектуры?
5. Что гарантирует декоратор @abstractmethod на этапе инициализации экземпляра класса?

Литература к лекции

1. **Мартин Р.** Чистая архитектура. – СПб.: Питер, 2021. (Глава 5: Объектно-ориентированное программирование: Полиморфизм).
2. **Рамальо Л.** Python. К вершинам мастерства. – 2-е изд. – СПб.: Питер, 2023. (Глава 13: Интерфейсы, протоколы и статический дак-тайпинг).
3. **PEP 544** — Protocols: Structural subtyping (static duck typing).